

Architecture

How the code is laid out, and the conventions that hold it together.

For *what the software does* — statuses, roles, TAT, the whitelist rules — see [domain rules](#). For running it, [local setup](#). For deploying it, [operations](#).

1 At a glance

A Next.js App Router application over Postgres, with no ORM. Server components read through repository functions and pass data into client components, which use TanStack Query for subsequent refetch and mutation. Business rules live in pure functions that never touch I/O; SQL lives in repositories that take a Kysely instance.

Two external services sit behind interfaces with two implementations each — a stub for development and the real thing for production. Nothing upstream knows which is in use.

2 Stack

Framework	Next.js 16.2.12, App Router, Turbopack
Language	TypeScript, React 19.2.4
Database	Postgres 17, Kysely — typed query builder, raw SQL where clearer. No ORM
Schema types	kysely-codegen generates src/modules/db/types.d.ts . Never hand-edited
Validation	zod v4, in wire.ts files at the request boundary
Auth	HS256 JWT via jose , in an httpOnly cookie
Email	AWS SES v2 (@aws-sdk/client-sesv2), templates in React Email
Styling	Tailwind v4 — CSS-first, no JS config
Components	shadcn/ui on Base UI (not Radix), base-nova style
Data fetching	TanStack Query v5
Charts	Recharts, wrapped by ui/chart.tsx
Animation	motion (Framer Motion's successor)
Icons	lucide-react — a deliberate substitution for missing Figma icon exports
Tests	Vitest, unit and integration in separate configs
Lint/format	Biome 2.2.0. No ESLint, no Prettier

3 Layering

```
src/app      → src/modules, src/lib, src/components
src/components → src/lib
src/modules  → src/modules/db, src/lib
```

Nothing lower reaches upward. Two things are worth knowing about how this is enforced:

No lint rule enforces it. There is no path-scoped override in `biome.json` restricting `next/*` imports from `src/lib` or `src/modules`. The boundary is convention, verified by inspection — and in practice it holds: `src/lib` has zero `next` imports, and `src/modules` has exactly one, `auth/service.ts`, whose `getSessionUser()` deliberately needs `next/headers`.

`src/middleware.ts` reads the JWT directly rather than calling `getSessionUser()`. It runs on the edge runtime, which cannot load Kysely or `pg`, so it calls `readSessionToken` and nothing that would pull a repository into the edge bundle. This constraint is why the file must not become `proxy.ts` by rename — see [operations §7](#).

The three kinds of file inside `modules/`

Kind	Contract	Examples
Pure rules	No I/O, no React, no <code>next/*</code>	<code>auth/roles.ts</code> , <code>reservations/decision.ts</code> , <code>reservations/types.ts</code> , <code>drivers/workload.ts</code> , <code>roster/types.ts</code> , <code>audit/changes.ts</code>
Repository	Takes <code>Kysely<DB></code> or <code>Transaction<DB></code> , owns SQL	<code>auth/repo.ts</code> , <code>admins/repo.ts</code> , <code>reservations/repo.ts</code> , <code>reservations/write.ts</code> , <code>roster/events.ts</code>
Service	Orchestrates the two above	<code>auth/service.ts</code> , <code>email/service.ts</code> , <code>email/dispatch.ts</code>

Pure-rule files are the ones worth reading first. They are where the domain lives, and they are trivially testable.

4 Directory structure

```
src/  
  app/  
    (admin)/      dashboard, master-list, calendar, reports,  
                  drivers, audit-logs, roster          – 7 pages  
    (auth)/       login, admin/login, not-authorized    – 3 pages  
    (requestor)/  /, book, manage                      – 3 pages  
    api/          route handlers (see §7)  
    globals.css   the entire design token contract  
    providers.tsx QueryClientProvider, TooltipProvider, Toaster  
  components/    65 non-test files across 6 directories (see §9)  
  hooks/  
  lib/           env, result, api-error, db-url, tz, utils  
  modules/       9 domain modules (see §5)  
  middleware.ts   the edge route gate  
  test/          integration harness
```

5 Modules

Module	Responsibility	Entry points
<code>auth</code>	Identity, roles, sessions, the provider seam	<code>authenticate</code> , <code>getSessionUser</code> , <code>requireUser</code> , <code>requireAdmin</code> , <code>resolveRole</code> , <code>resolveSuperAdmin</code> , <code>selectAuthProvider</code>
<code>db</code>	Kysely client, generated types, migrations, seeds	<code>createDb</code> , <code>getDb</code>
<code>reservations</code>	The booking domain — the largest module	<code>listReservations</code> , <code>write.ts</code> transitions, <code>decision.ts</code> rules, <code>calendar.ts</code> , <code>dashboard-metrics.ts</code> , <code>export.ts</code>
<code>admins</code>	CRUD over <code>admin_whitelist</code> , plus the four guardrails	<code>listAdmins</code> , <code>isSuperAdmin</code> , <code>createAdmin</code> , <code>updateAdmin</code> , <code>setAdminActive</code>
<code>drivers</code>	Driver roster and weekly workload	<code>listDrivers</code> , <code>createDriver</code> , <code>updateDriver</code> , <code>setDriverActive</code> , <code>WEEKLY_HOURS_CAP</code>
<code>vans</code>	Van roster. Structurally identical to <code>drivers</code>	<code>listVans</code> , <code>createVan</code> , <code>updateVan</code> , <code>setVanActive</code>
<code>roster</code>	Shared roster machinery for the three above	<code>recordRosterEvent</code> , <code>diffFields</code> , <code>wire.ts</code> schemas, <code>affectedTrips</code>
<code>audit</code>	Read-only views over the two event trails	<code>listAuditEntries</code> , <code>listRosterEvents</code> , <code>parseChanges</code>
<code>email</code>	Outbound mail: transports, templates, the outbox dispatcher	<code>sendEmail</code> , <code>dispatchOutbox</code> , <code>adminRecipients</code> , <code>selectMailTransport</code>

`admins`, `drivers`, and `vans` are three near-identical create/patch/setActive modules, **deliberately not generalised**. Only `admins` needs guardrails — there is no single-point-of-failure capability at stake in a driver or a van — and collapsing them would put those guardrails behind an abstraction that the other two would have to opt out of.

`db` imports nothing from other modules at runtime. The exceptions are all non-runtime: the `seeds/` and `dev-seed/` scripts import `auth` and `reservations` types for fixture realism, and this module's own `*.int.test.ts` files import from `auth`, `reservations` and `roster` to assert against real constraints.

6 Data layer

`createDb(connectionString)` builds a `Kysely<DB>` over a `pg` Pool capped at 10 connections. `getDb()` caches one instance keyed off `env().DATABASE_URL`.

Tables

Eleven tables. Two of them are append-only event logs, and the distinction between them matters:

Table	Holds
<code>users</code>	One row per person who has signed in
<code>admin_whitelist</code>	Who may hold <code>admin_support</code> ; the <code>super_admin</code> flag; per-site notification routing
<code>drivers</code>	Driver roster. No longer holds van identity
<code>vans</code>	Van roster — <code>van_number</code> , <code>plate</code> (unique), <code>car_type</code> , <code>site</code> , <code>active</code>
<code>reservations</code>	One row per trip. Wide: mode, status, site, requestor snapshot, purpose, timing, locations, cost, driver-or-rental, van-or-rental, reasons, <code>version</code>
<code>reservation_passengers</code>	Named seats, unique per reservation and position
<code>reservation_events</code>	Append-only trail of reservation writes. Permits a null actor for system events
<code>roster_events</code>	Append-only trail of roster writes. Polymorphic over <code>drivers/vans/admin_whitelist</code> . <code>actor_name</code> and <code>actor_role</code> are NOT NULL — a person always causes a roster change — though the <code>actor_user_id</code> FK itself is nullable
<code>notification_events</code>	One row per notifiable event; the in-app spine
<code>notification_outbox</code>	The email queue — one row per outbound message
<code>reference_counters</code>	Per-year counter behind <code>VR-2026-000412</code> reference numbers

There is no **audit_log** table. The admin action log on `/audit-logs` is a *view over* `reservation_events`, filtered to `actor_role = 'admin_support'`. A second log would be a second thing to keep in sync. `roster_events` is a real, separate table because it has a different shape — polymorphic target, non-nullable actor.

Migrations

Thirteen, in order. Each row names the constraint or column it introduced, so a schema question can be traced back to the migration that caused it.

#	Migration	Did
1	<code>create_auth_tables</code>	<code>users</code> , <code>admin_whitelist</code> ; site check, identity check, unique <code>lower(email)</code>
2	<code>identity_hardening</code>	Unique <code>lower(domain_id)</code> on <code>users</code> ; whitelist email index becomes <code>lower(trim(email))</code> , closing a whitespace-duplicate hole
3	<code>create_reservation_tables</code>	<code>drivers</code> (with van columns, later removed), <code>reservations</code> with its full CHECK set, <code>reservation_passengers</code> , <code>reservation_events</code> , <code>reference_counters</code>
4	<code>create_notification_tables</code>	<code>notification_events</code> , <code>notification_outbox</code> with <code>sent ⇔ (sent_at ^ provider_message_id)</code>
5	<code>create_vans_table</code>	<code>vans</code> as its own table
6	<code>add_reservation_van_and_rental_columns</code>	<code>assigned_van_id</code> + five <code>rental_*</code> columns; roster-or-rental exclusivity CHECKs for both sides
7	<code>split_van_from_drivers</code>	Drops <code>van_number/plate</code> from <code>drivers</code> . Makes <code>shift</code> nullable and drops its 3-value CHECK
8	<code>add_van_assignment_events</code>	Admits <code>van_assigned</code> / <code>van_reassigned</code> event types
9	<code>reservation_details</code>	<code>reservations.details</code> , NOT NULL with a non-blank CHECK, backfilled
10	<code>remap_retired_purposes</code>	Data-only: five retired purpose strings → replacements. <code>down()</code> is an intentional no-op
11	<code>add_reassigned_status</code>	Admits <code>approved_reassigned</code> , widens the approval CHECKs
12	<code>audit_log_index</code>	<code>(created_at desc, id desc)</code> on <code>reservation_events</code> — the existing <code>reservation_id</code> -leading index cannot serve a whole-table keyset scan

#	Migration	Did
13	<code>roster_management</code>	<code>admin_whitelist.super_admin</code> (default <code>false</code> , so no existing row is silently granted it); <code>roster_events</code> ; a matching keyset index

Migration 6's constraints are the ones most likely to block a deploy against real data. See [operations §3](#).

Trips and rentals

One `reservations` row is one trip. There is no parent booking table; a wizard submission inserts up to `MAX_TRIPS_PER_SUBMISSION` (10) rows, each with its own reference number.

Two ride modes, enforced by `reservations_mode_shape_check`:

- **pickup** — point to point. Has a dropoff location; no `end_at`, tower head, vendor, or cost.
- **standby** — holds a van and driver for a block. Requires `end_at` and an approving tower head; no dropoff location.

A trip's driver and van are each **either** a roster reference **or** a one-off rental typed in at approval time, never both. Modelled as discriminated unions in code (`source: "roster" | "rental"`) and as mutually-exclusive CHECK constraints in the database. See [domain §7](#).

7 API and error handling

Routes

Path	Methods
<code>/api/auth/login · /logout</code>	POST
<code>/api/auth/session</code>	GET
<code>/api/auth/verify-domain</code>	POST
<code>/api/associates/lookup</code>	POST
<code>/api/reservations</code>	GET, POST
<code>/api/reservations/[id]</code>	GET, PATCH
<code>/api/reservations/[id]/cancel</code>	POST
<code>/api/drivers · /api/vans · /api/admins</code>	GET, POST
<code>/api/drivers/[id] · /api/vans/[id] · /api/admins/[id]</code>	PATCH
<code>/api/drivers/[id]/affected · /api/vans/[id]/affected</code>	GET
<code>/api/audit-logs</code>	GET
<code>/api/internal/dispatch-outbox</code>	GET
<code>/api/dev/email · /[template] · /probe</code>	GET

One **PATCH** per resource handles both a field edit and an activation change. Splitting them would put one resource's state behind two URLs. The two cases are distinguished by trying two `.strict()` schemas in sequence, so a body mixing both matches neither and 422s.

`/api/internal/dispatch-outbox` is guarded by a shared secret header, not session auth — it is called by a scheduler, and it is excluded from the middleware matcher along with the rest of `/api`.

`/api/dev/*` routes 404 in production.

The error model

`Result<T, E>` in `src/lib/result.ts` — `{ok: true, value}` or `{ok: false, error}`. Repositories and rules return it; route handlers convert.

`src/lib/api-error.ts` maps error codes to status:

Code	Status
<code>VALIDATION_FAILED, DRIVER_REQUIRED</code>	422
<code>INVALID_TRANSITION, VERSION_CONFLICT</code>	409
<code>NOT_AUTHENTICATED</code>	401
<code>FORBIDDEN</code>	403
<code>NOT_FOUND</code>	404
<code>RATE_LIMITED</code>	429
<code>SERVICE_UNAVAILABLE</code>	503

All ten codes in `ERROR_STATUS`. The last three are provider-facing: `SERVICE_UNAVAILABLE` and `RATE_LIMITED` come from the upstream auth provider or SES, and `EMAIL_REJECTED` marks a permanently refused message — see [email §8](#).

`errorResponse(code, message, details?)` emits `{ error: { code, message, details? } }` at the mapped status.

Module error types are `Extracted` from that map rather than declared separately, so removing a code breaks compilation at every consumer instead of letting the two drift. `EmailFailure` and `AuthFailure` both work this way.

Wire schemas

Three `wire.ts` files — `reservations/`, `roster/`, `audit/` — own zod parsing for their module's bodies and query params, kept separate from the pure rules so a structural error ("wrong shape") stays distinguishable from a business-rule error.

`roster/wire.ts`'s PATCH and activation schemas are `.strict()`; its three create schemas are not.

Without `.strict()` zod strips unknown keys silently, so a body with a typo'd field name half-applies — which is why the patch paths have it (`driverPatchSchema:47`, `activeSchema:62`, `vanPatchSchema:75`, `adminPatchSchema:133`) and why `driverCreateSchema`, `vanCreateSchema` and `adminCreateSchema` arguably should. PATCH schemas additionally refuse an empty object, so an empty patch cannot write a no-op `updated` audit event.

8 Auth

The provider seam

`AuthProvider` has three methods: `identify`, `verifyDomain`, and `lookupName(domainId)`. Two implementations — `StubAuthProvider` for development, `CgsAuthProvider` for the real CGS Associate Authentication API — selected by `AUTH_MODE`. Nothing upstream of the interface knows which is live.

`lookupName` deliberately returns only `{name}`, never contact details, because it backs the booking wizard's passenger autofill and must not disclose other associates' information.

Sign-in

`authenticate()` takes a `portal` of `"requestor"` or `"admin"`. There is no separate admission allowlist — the CGS directory authenticates only active employees, so a successful `identify` is the admission check. `admin_whitelist` decides capability, and only at the admin door. See [domain §2](#) for the full resolution order.

A refused admin attempt fails **before** `upsertUser` runs, so it leaves no `last_login_at` trace.

Sessions

`vr_session`, an HS256 JWT via `jose`, `httpOnly`, with a required `exp` claim. `SESSION_TTL_SECONDS` is 8 hours.

The payload carries `superAdmin`, which **defaults to `false` on parse** so that cookies signed before the field existed still validate. Without that default, adding the field would have signed out every logged-in user on deploy.

Two enforcement points

1. `src/middleware.ts` — edge, page navigation only. `resolveRedirect` is a pure function, unit-tested. `ADMIN_PREFIXES` covers all seven admin routes.
2. `requireUser()` / `requireAdmin()` — Node runtime, called explicitly by every route handler, because `/api/*` is excluded from the middleware matcher.

`requireAdmin()` is an explicit allowlist on the literal string `"admin_support"`, so a future third role is denied by default.

Capability is re-read, never trusted from the cookie

`SessionUser.superAdmin` is documented as **cosmetic**. **Nothing authorizes on it, and nothing reads it.**

A capability baked into a signed cookie lags the database in *both* directions: it survives a revocation, and it misses a grant. Trusting it produced a real bug during development — a freshly promoted holder saw the Admins tab (because the page does its own database read) and got a 403 on every write behind it, for the life of the cookie.

So every consumer re-reads: `isSuperAdmin(db, identity) → resolveSuperAdmin(identity, await listActiveWhitelist(db))`. The `/api/admins` gate does it per request. `/api/audit-logs` does it to decide whether whitelist-target entries are visible. `/roster`'s page does it rather than reading `user.superAdmin`.

`isSuperAdmin` matches on the **full identity** — Domain ID *and* email. Matching on Domain ID alone previously reopened a self-lockout hole.

9 Frontend

Rendering model

Every page is a server component. All 13 pages across the three route groups; zero carry `"use client"`. They fetch through repositories directly and pass results as props into client views.

Of the 81 files in `src/components/`, 43 are client components — forms, drawers, dialogs, charts, the animated login van. The pattern is a server shell with client leaves: `login-hero.tsx` stays on the server so its wedge and gradient ship no JavaScript, and only its child `login-van.tsx` is a client component.

TanStack Query is configured with `staleTime: 30_000`, `refetchOnWindowFocus: false`, and **mutations are never retried** — replaying an approve or a cancel is worse than surfacing the failure. It is used in ten components; query keys are centralised per module.

Every page checks its own session — deliberately

There is **no `layout.tsx` in any route group**. Every admin and requestor page repeats its own `getSessionUser()` and redirect, each with a comment saying why:

A layout does not re-execute on client-side navigation between sibling routes, so a layout-only check would be a fail-open.

This is a convention, not duplication to be cleaned up. A new admin page must copy the block.

The token contract

Tailwind v4 has no JS config. Every token is declared in `src/app/globals.css` inside `@theme`.

Group	Tokens
Brand	<code>brand</code> #5009b5, <code>primary</code> #7142ff, <code>navy</code> , <code>plum</code>
Requestor	<code>requestor-text-accent</code> , <code>requestor-blob-highlight</code> , <code>requestor-field-line</code> , <code>requestor-field-line-strong</code>
Admin	<code>admin-accent</code> #00650a, <code>admin-panel</code> , <code>admin-heading</code> , <code>admin-blob-highlight</code>
Hero	<code>hero-glow</code> , <code>hero-base</code> , <code>hero-deep</code> , <code>hero-accent</code> , <code>hero-shadow</code> , <code>hero-scrim</code>
Surfaces	<code>brand-tint</code> , <code>brand-wash</code> , <code>site-iloilo-tint/-ink</code> , <code>site-manila-tint/-ink</code>
Grey / state	<code>gray-1...gray-6</code> , <code>error</code> , <code>success</code> , <code>error-tint</code> , <code>error-tint-border</code> , <code>success-tint</code>
Type	<code>--font-sans</code> ; <code>--text-h1/h2/h3/body/sm/xs</code> , <code>--text-display</code> (5.25rem, requestor hero only)
Radii	<code>pill</code> 9999px, <code>field</code> 0.5rem, <code>card</code> 1rem

Two further layers sit on top: a shadcn semantic layer inside `@theme`, and a bare `:root` alias layer of 19 entries feeding `color-mix()` and inline `style` props.

`--color-border` and `--color-input` map to `gray-3`, not `gray-4` — `gray-4` is 1.6:1 and fails WCAG 1.4.11's 3:1 minimum for component boundaries.

The design is light-only, and one line enforces that

```
@custom-variant dark (&:is(.dark *));
```

The generated shadcn primitives ship `dark:` utilities. No `.dark` class is ever rendered, and this line is the only thing preventing an OS in dark mode from applying them.

It must stay after all three `@imports` — CSS requires imports to precede other at-rules, and Biome's `noInvalidPositionAtImportRule` will flag it otherwise. Do not move it, and do not add `dark:` variants.

Verifying a token is live — grep is not enough

Three separate mechanisms make a token look present while doing nothing:

1. **Pruning.** Tailwind emits `@theme` custom-property *declarations* whether used or not, but generates the **utility-class selector** only if something references it. `--text-h2: 3rem` is in the compiled CSS while `.text-h2{...}` is not. The declaration is not proof.
2. **Baking.** A hand-written `color-mix()` inside a custom `@utility` gets fully evaluated by Lightning CSS when its inputs are static, compiling to a flat literal with no `var()` left. Editing the token then changes nothing. Prefer Tailwind's alpha modifier (`to-hero-scrim/72`).
3. **twMerge dropping.** `cn()` is plain `twMerge(clsx(...))` with no `extendTailwindMerge` config, so tailwind-merge does not know this project's custom font-size tokens and classifies them by suffix as text colours. `cn("text-body", "text-gray-1")` silently drops `text-body`.

So: `rm -rf .next && pnpm build`, then grep the compiled CSS for the **selector** `.bg-admin-accent{}`, not the declaration. And for case 3, check the rendered HTML — the class is in the CSS file, it just never reaches the element.

A declared token is not a used token. `--color-admin-text-accent` is declared, contrast-documented, and referenced nowhere. `grep -rn <token> src/` before trusting any comment about where it "ships".

Variant theming

Admin and requestor surfaces differ through modules of **literal** Tailwind class-string constants — never assembled at runtime, so Tailwind's scanner can see them. `login-theme.ts` keys an accent record by variant; `admin-theme.ts` and `requestor-theme.ts` hold per-surface strings.

The requestor variant uses the `hero-*` gradients; admin uses flat `bg-admin-panel` and `bg-admin-accent`, because no admin gradient design exists to transcribe.

`FOCUS_RING` is project-wide rather than per-variant, because the Figma design draws no focus state on any screen — a WCAG 2.4.7 gap closed in code.

Components

Counts are non-test files; they sum to the 65 in §4.

Directory	Files	Contents
ui/	7	Base UI primitives: button, card, chart, input, label, sonner, tooltip
auth/	10	login-form, login-hero, login-van (Motion drive-in, reduce-motion aware), domain-id-input (7-box, fieldset + per-box labels), password-field, sign-out-button
shells/	2	requestor-shell; admin-shell — 256px sticky sidebar, topbar, working area
common/	4	hint, rental-tag, row-menu, status-chip — shared across both surfaces
admin/	30	admin-nav, admin-topbar, plus audit/, calendar/, dashboard/, drivers/, master-list/, reports/, roster/, trip-drawer/
requestor/	19	hero-*, home-view, manage/, and the 4-step wizard/

Notable pieces: [roster/roster-view.tsx](#) (three tabs, all four forms have paired tests), [audit/audit-view.tsx](#) (cursor-paginated via [useInfiniteQuery](#)), [trip-drawer/](#) (the per-reservation approve/decide panel, with a [useTripDrawer\(\)](#) host consumed by the master list).

The topbar bell is deliberately inert — [unreadCount](#) is hardcoded to [0](#) and clicking it toasts "Notifications aren't wired up yet."

Dates and times go through one file

[src/lib/tz.ts](#) is the only place in the codebase permitted to format a date or time, enforced by a drift-guard test that scans [src/](#) for [toLocaleDateString](#), [toLocaleTimeString](#), [toLocaleString](#), and [Intl.DateTimeFormat](#) outside that file.

10 Conventions that must not regress

Most of these exist because their absence caused a real bug; the rest are preventive. The second column names what actually stops a regression — and is honest where the answer is nothing.

Convention	Enforced by
Roster events are written in the transaction that made the change	<code>recordRosterEvent</code> takes <code>Transaction<DB></code> , never <code>Kysely<DB></code> — the type system
Every page checks its own session	Convention plus a comment on each page. No route-group layout exists to regress into
Date formatting lives in <code>tz.ts</code>	A drift-guard test that greps <code>src/</code>
Roster PATCH bodies reject unknown keys	<code>.strict()</code> on the patch and activation schemas in <code>roster/wire.ts</code> . The three create schemas lack it — unknown keys on a POST are stripped, not rejected
Guardrails cannot be routed around	They live inside the repo layer, inside the transaction
<code>super_admin</code> is never read from the cookie	The cookie field has zero readers; every consumer calls <code>isSuperAdmin</code>
The design stays light-only	<code>@custom-variant dark</code> + Biome's import-position rule
Error codes cannot drift from HTTP status	Module failure types are <code>Extracted</code> from <code>ERROR_STATUS</code>
No <code>next/*</code> in <code>src/lib</code>	Nothing. Convention only — a lint rule would be worth adding

See also

- [Domain rules](#) — the business rules
- [Local setup](#) — local development
- [Operations](#) — environment, deploy, runbook
- [Email and notifications](#) — the notification pipeline and the templates